

Assignment 2

Very busy expressions

Dominio

Insiemi di espressioni.

Direzione

La definizione di very busy expression parla dei percorsi da un punto p del codice al blocco exit, perciò l'analisi è backward.

Funzione di trasferimento

Per le singole istruzioni:

Data l'istruzione I, la funzione di trasferimento è:

$$f_I(x) = (x - \text{Kill}[I]) \cup \text{Use}[I]$$

dove $\text{Kill}[I]$ è:

- se I è un assegnamento, un insieme che contiene tutte le espressioni che usano la variabile a sinistra dell'assegnamento
- l'insieme vuoto altrimenti

e $\text{Use}[I]$ è un insieme che contiene l'espressione usata da I se tale espressione esiste altrimenti l'insieme vuoto.

Per i basic block:

Dato il basic block B, la funzione di trasferimento è:

$$f_B(x) = (x - \text{Kill}[B]) \cup \text{Use}[B]$$

dove $\text{Kill}[B]$ contiene l'espressione E se:

- essa non viene usata da nessuna istruzione del BB e almeno una delle variabili usate dall'espressione viene definita nel BB.
- essa viene usata da almeno un'istruzione del BB e almeno una delle variabili usate dall'espressione viene definita nel BB prima del primo uso dell'espressione.

e $\text{Use}[B]$ contiene l'espressione E se essa viene usata da almeno un'istruzione del BB e nessuna delle variabili usate dall'espressione viene definita nel BB prima del primo uso dell'espressione.

Operatore di meet ($\wedge$)

Secondo la definizione di very busy expression:

- Un'espressione è very busy in un punto p se, **indipendentemente dal percorso preso da p** , l'espressione viene usata prima che uno dei suoi operandi venga definito.
- Un'espressione $a+b$ è very busy in un punto p se $a+b$ è valutata **in tutti i percorsi da p a EXIT** e non c'è una definizione di a o b lungo tali percorsi

Le parti evidenziate rendono chiaro il fatto che l'operatore di meet deve essere l'intersezione.

Boundary condition

La boundary condition è $\text{in}[\text{EXIT}] = \emptyset$ perché tra il punto appena prima del blocco di exit e il blocco di exit non c'è codice quindi neanche espressioni usate.

Inizializzazione punti interni

Gli insiemi relativi ai punti interni vanno inizializzati all'elemento neutro dell'operatore di meet. Nel nostro caso l'operatore di meet è l'intersezione, il cui elemento neutro è l'insieme universo, che in questo caso è l'insieme di tutte le espressioni usate da una qualche istruzione nel codice.

Riassunto

Dominio	Insiemi di espressioni
Direzione	Backward: $\text{in}[b] = f_b(\text{out}[b])$ $\text{out}[b] = \wedge \text{in}[\text{succ}(b)]$
Funzione di trasferimento	$f_B(x) = (x - \text{Kill}[B]) \cup \text{Use}[B]$
Operatore di meet ($\wedge$)	Intersezione
Boundary condition	$\text{in}[\text{EXIT}] = \emptyset$
Initial interior points	$\text{in}[B] = U$ (insieme universo)

Esecuzione dell'algoritmo

	Iterazione 1	
	out[B]	in[B]
BB8	-	{}
BB7	{}	{a-b}
BB6	{a-b}	{}
BB5	{}	{b-a}
BB4	{}	{a-b}
BB3	{a-b}	{a-b, b-a}
BB2	{b-a}	{b-a, a!=b}
BB1	{b-a, a!=b}	-

Non essendoci cicli nel CFG, è sufficiente un'unica iterazione.

Dominator analysis

Dominio

Insiemi di basic block.

Direzione

La definizione di dominanza parla dei percorsi dal blocco entry a un basic block Y, perciò l'analisi è forward.

Funzione di trasferimento

La dominator analysis non associa insiemi ai punti del codice, ma ai basic block, quindi non associa ad ogni BB due insiemi in[B] e out[B] ma un solo insieme dom[B] e non ha una funzione di trasferimento.

Operatore di meet (^)

Secondo la definizione di dominanza di un BB su un altro:

*In un CFG diciamo che un nodo X domina un altro nodo Y se il nodo X appare **in ogni percorso** del grafo che porta dal blocco ENTRY al blocco Y*

La parte evidenziata rende chiaro il fatto che l'operatore di meet deve essere l'intersezione.

Boundary condition

La boundary condition è $\text{dom}[\text{ENTRY}] = \{\text{ENTRY}\}$ perché per definizione un nodo domina sé stesso e il blocco entry non può essere dominato da un altro nodo essendo il primo.

Inizializzazione punti interni

Gli insiemi relativi ai nodi vanno inizializzati all'elemento neutro dell'operatore di meet. Nel nostro caso l'operatore di meet è l'intersezione, il cui elemento neutro è l'insieme universo, che in questo caso è l'insieme di tutti i nodi del CFG.

Riassunto

Dominio	Insiemi di nodi/basic block
Direzione	Forward: $\text{dom}[b] = \bigwedge \text{dom}[\text{pred}(b)] \cup \{b\}$
Funzione di trasferimento	-
Operatore di meet ($\wedge$)	Intersezione
Boundary condition	$\text{dom}[\text{ENTRY}] = \{\text{ENTRY}\}$
Initial interior points	$\text{dom}[B] = U$ (insieme universo)

Esecuzione dell'algoritmo

	Iterazione 1
	$\text{dom}[B]$
A	$\{A\}$
B	$\{A, B\}$
C	$\{A, C\}$
D	$\{A, C, D\}$
E	$\{A, C, E\}$
F	$\{A, C, F\}$
G	$\{A, G\}$

Non essendoci cicli nel CFG, è sufficiente un'unica iterazione.

Constant propagation

Dominio

Insiemi di coppie <variabile, valore costante>.

Direzione

Quali variabili hanno valore costante in un certo punto del codice dipende dalle istruzioni precedenti, quindi la direzione è forward.

Funzione di trasferimento

Per le singole istruzioni:

Dato l'istruzione I, la funzione di trasferimento è:

$$f_I(x) = (x - \text{Kill}[I]) \cup \text{Def}[I]$$

dove $\text{Kill}[I]$ è:

- se I è un assegnamento, l'insieme di tutte le coppie $\langle v, c \rangle$ dove v è la variabile a sinistra dell'assegnamento
- l'insieme vuoto altrimenti

e $\text{Def}[I]$ è:

- se I è un assegnamento la cui parte destra ha valore costante, un insieme che contiene la coppia $\langle v, c \rangle$ dove v è la variabile a sinistra dell'assegnamento e c è il valore costante della parte destra.
- L'insieme vuoto altrimenti

Per i basic block:

Dato il basic block B, la funzione di trasferimento è:

$$f_B(x) = (x - \text{Kill}[B]) \cup \text{Def}[B]$$

dove:

- $\text{Kill}[B]$ contiene le coppie della forma $\langle v, c \rangle$ se esiste nel blocco B almeno un assegnamento alla variabile v e l'ultimo di essi non è un assegnamento a un valore costante.
- $\text{Def}[B]$ contiene la coppia $\langle v, c \rangle$ se esiste nel blocco B almeno un assegnamento alla variabile v e l'ultimo di essi è un assegnamento al valore costante c.

Operatore di meet (^)

Secondo la definizione di constant propagation:

Se abbiamo la coppia $\langle x, c \rangle$ al nodo n, significa che x è garantito avere il valore c **ogni volta che n viene raggiunto** durante l'esecuzione del programma.

Ogni volta che n viene raggiunto = qualsiasi percorso venga fatto per arrivare a n $\Rightarrow$ intersezione

Boundary condition

La boundary condition è $\text{out}[\text{ENTRY}] = \{\}$ perché non essendo stato eseguito codice, appena dopo il blocco entry non ci sono variabili con valore costante

Inizializzazione punti interni

Gli insiemi relativi ai punti interni vanno inizializzati all'elemento neutro dell'operatore di meet. Nel nostro caso l'operatore di meet è l'intersezione, il cui elemento neutro è l'insieme universo.

Riassunto

Dominio	Insiemi di coppie <variabile, valore costante>
Direzione	Forward: $\text{out}[b] = f_b(\text{in}[b])$ $\text{in}[b] = \wedge \text{out}[\text{prev}(b)]$
Funzione di trasferimento	-
Operatore di meet ($\wedge$)	Intersezione
Boundary condition	$\text{out}[\text{ENTRY}] = \{\}$
Initial interior points	$\text{out}[B] = U$ (insieme universo)

Esecuzione dell'algoritmo

	Iterazione 1	
	$\text{in}[B]$	$\text{out}[B]$
entry	-	$\{\}$
$k = 2$	$\{\}$	$\{<k, 2>\}$
if	$\{<k, 2>\}$	$\{<k, 2>\}$
$a = k + 2$	$\{<k, 2>\}$	$\{<k, 2>, <a, 4>\}$
$x = 5$	$\{<k, 2>, <a, 4>\}$	$\{<k, 2>, <a, 4>, <x, 5>\}$
$a = k * 2$	$\{<k, 2>\}$	$\{<k, 2>, <a, 4>\}$
$x = 8$	$\{<k, 2>, <a, 4>\}$	$\{<k, 2>, <a, 4>, <x, 8>\}$
$k = a$	$\{<k, 2>, <a, 4>\}$	$\{<k, 4>, <a, 4>\}$
while	$\{<k, 4>, <a, 4>\}$	$\{<k, 4>, <a, 4>\}$
$b = 2$	$\{<k, 4>, <a, 4>\}$	$\{<k, 4>, <a, 4>, <b, 2>\}$
$x = a + k$	$\{<k, 4>, <a, 4>, <b, 2>\}$	$\{<k, 4>, <a, 4>, <b, 2>, <x, 8>\}$
$y = a * b$	$\{<k, 4>, <a, 4>, <b, 2>, <x, 8>\}$	$\{<k, 4>, <a, 4>, <b, 2>, <x, 8>, <y, 8>\}$
$k++$	$\{<k, 4>, <a, 4>, <b, 2>, <x, 8>, <y, 8>\}$	$\{<k, 5>, <a, 4>, <b, 2>, <x, 8>, <y, 8>\}$
$\text{print}(a + x)$	$\{<k, 4>, <a, 4>\}$	$\{<k, 4>, <a, 4>\}$
exit	$\{<k, 4>, <a, 4>\}$	-

	Iterazione 2	
	in[B]	out[B]
entry	-	{}
k = 2	{}	{<k, 2>}
if	{<k, 2>}	{<k, 2>}
a = k + 2	{<k, 2>}	{<k, 2>, <a, 4>}
x = 5	{<k, 2>, <a, 4>}	{<k, 2>, <a, 4>, <x, 5>}
a = k * 2	{<k, 2>}	{<k, 2>, <a, 4>}
x = 8	{<k, 2>, <a, 4>}	{<k, 2>, <a, 4>, <x, 8>}
k = a	{<k, 2>, <a, 4>}	{<k, 4>, <a, 4>}
while	{<a, 4>}	{<a, 4>}
b = 2	{<a, 4>}	{<a, 4>, <b, 2>}
x = a + k	{<a, 4>, <b, 2>}	{<a, 4>, <b, 2>}
y = a * b	{<a, 4>, <b, 2>}	{<a, 4>, <b, 2>, <y, 8>}
k++	{<a, 4>, <b, 2>, <y, 8>}	{<a, 4>, <b, 2>, <y, 8>}
print(a + x)	{<a, 4>}	{<a, 4>}
exit	{<a, 4>}	-

	Iterazione 3	
	in[B]	out[B]
entry	-	{}
k = 2	{}	{<k, 2>}
if	{<k, 2>}	{<k, 2>}
a = k + 2	{<k, 2>}	{<k, 2>, <a, 4>}
x = 5	{<k, 2>, <a, 4>}	{<k, 2>, <a, 4>, <x, 5>}
a = k * 2	{<k, 2>}	{<k, 2>, <a, 4>}
x = 8	{<k, 2>, <a, 4>}	{<k, 2>, <a, 4>, <x, 8>}
k = a	{<k, 2>, <a, 4>}	{<k, 4>, <a, 4>}
while	{<a, 4>}	{<a, 4>}
b = 2	{<a, 4>}	{<a, 4>, <b, 2>}
x = a + k	{<a, 4>, <b, 2>}	{<a, 4>, <b, 2>}
y = a * b	{<a, 4>, <b, 2>}	{<a, 4>, <b, 2>, <y, 8>}
k++	{<a, 4>, <b, 2>, <y, 8>}	{<a, 4>, <b, 2>, <y, 8>}
print(a + x)	{<a, 4>}	{<a, 4>}
exit	{<a, 4>}	-

Le iterazioni due e 3 sono uguali, quindi l'algoritmo ha raggiunto la convergenza.